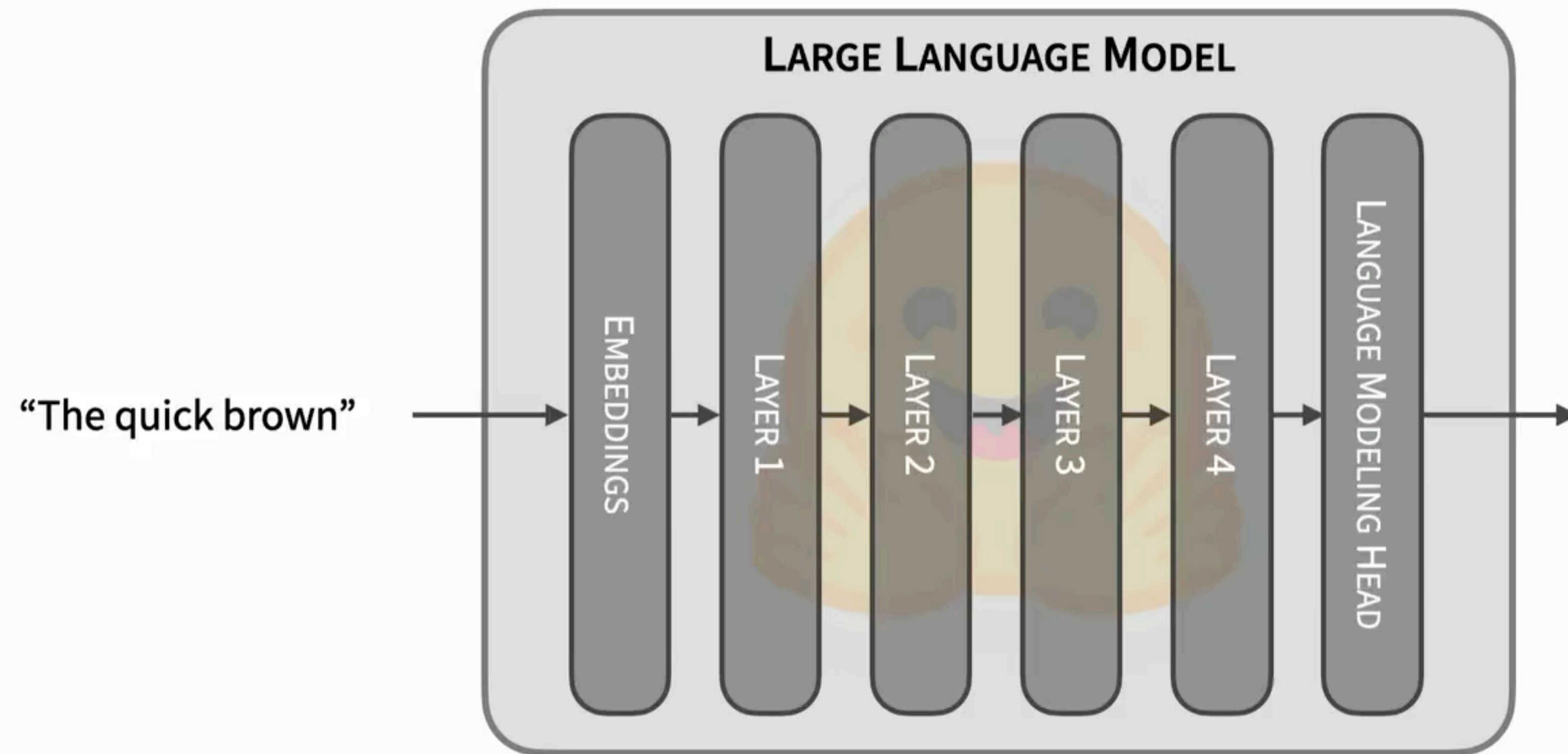


Decoding

NLP: Fall 2025

Anoop Sarkar

Causal Language Models



https://huggingface.co/docs/transformers/llm_tutorial

Causal Language Models



"Autoregressive generation iteratively selects the next token from a probability distribution to generate text"

Causal LMs: Common Pitfalls

- **Generated output is too short/long:** LM may require further tuning, also asking for more tokens can help
- **Incorrect generation mode:** greedy decoding or sampling? Which is better depends on your task
- **Wrong padding side:** you may need to pad the prompt text on the left to ensure that the input is the same size as the training phase of the LM.
- **Wrong prompt:** this is tricky and has produced a whole industry of "prompt engineering"

https://huggingface.co/docs/transformers/llm_tutorial

c.f. for code
samples

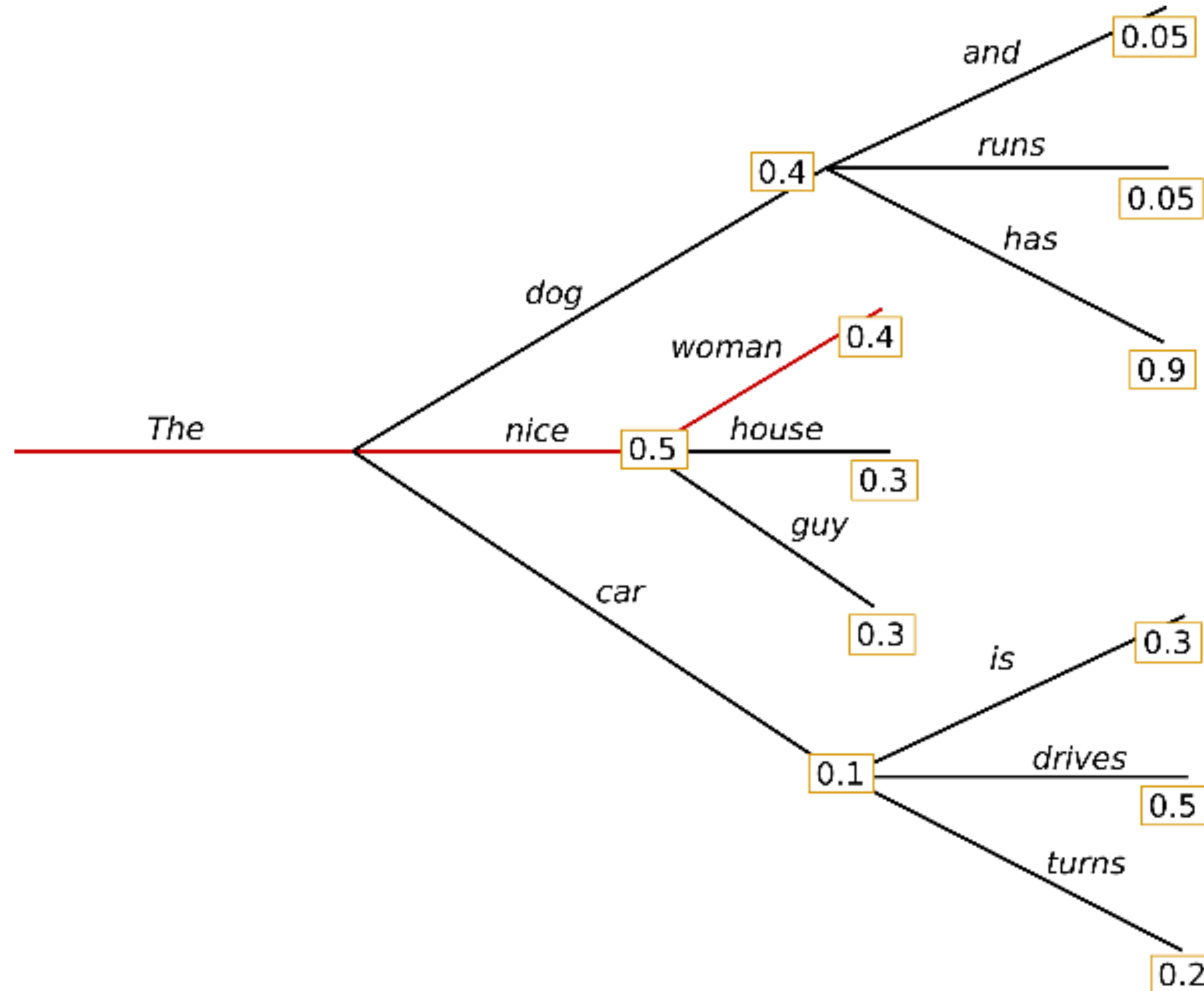
Decoding methods

<https://huggingface.co/blog/how-to-generate>

$$P(w_{1:T}|W_0) = \prod_{t=1}^T P(w_t|w_{1:t-1}, W_0) , \text{with } w_{1:0} = \emptyset,$$

- W_0 is the initial context word sequence (aka the "prompt")
- The length T of the word sequence is determined on-the-fly
- T is determined by the generation of the end-of-sentence EOS also known as the `<|endof text|>` token
- The EOS token is produced like the other tokens from $P(w_t | w_{1:t-1}, W_0)$

Greedy Decoding



("The", "nice", "woman")
having an overall
probability of
 $0.5 \times 0.4 = 0.2$

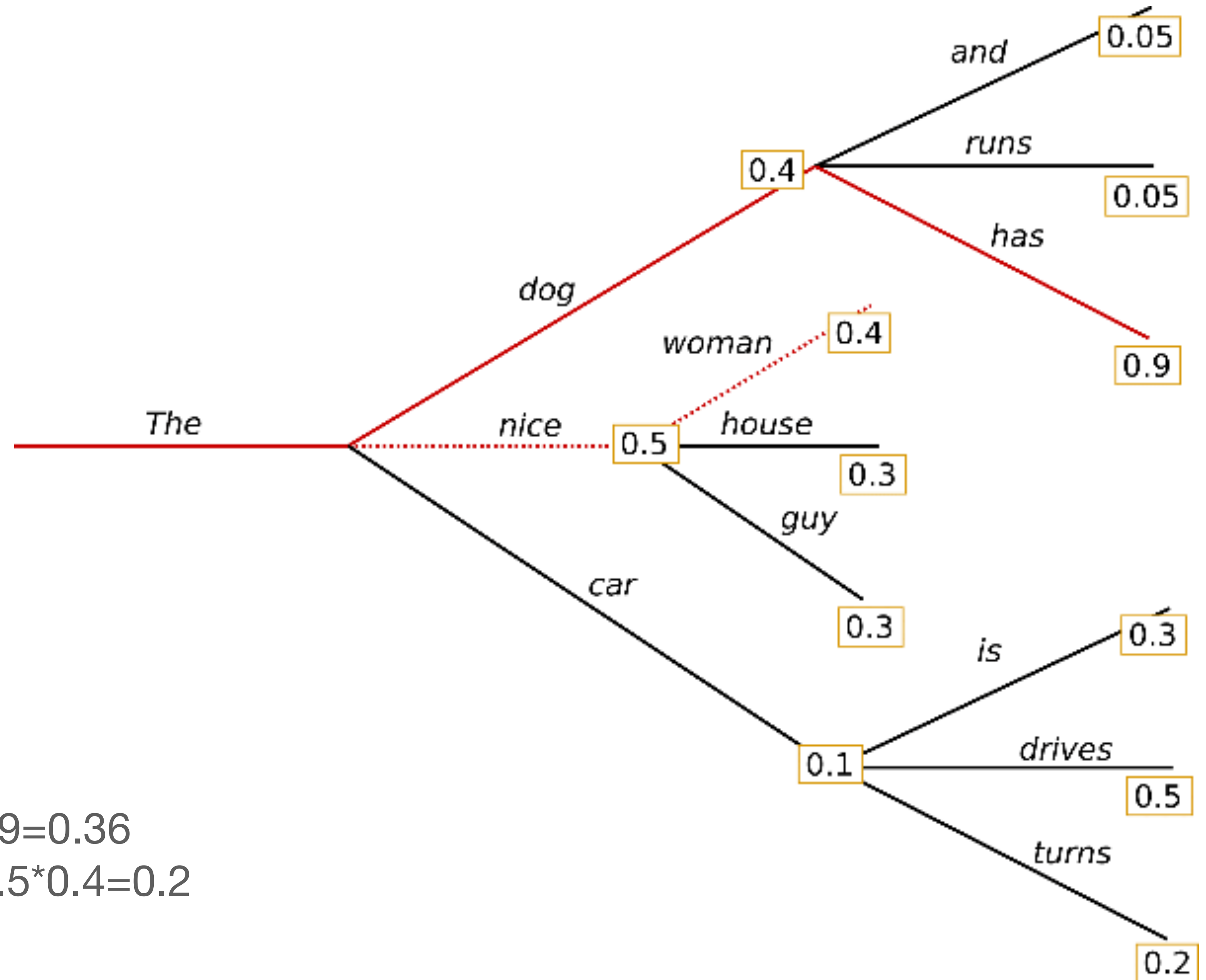
Beam Search

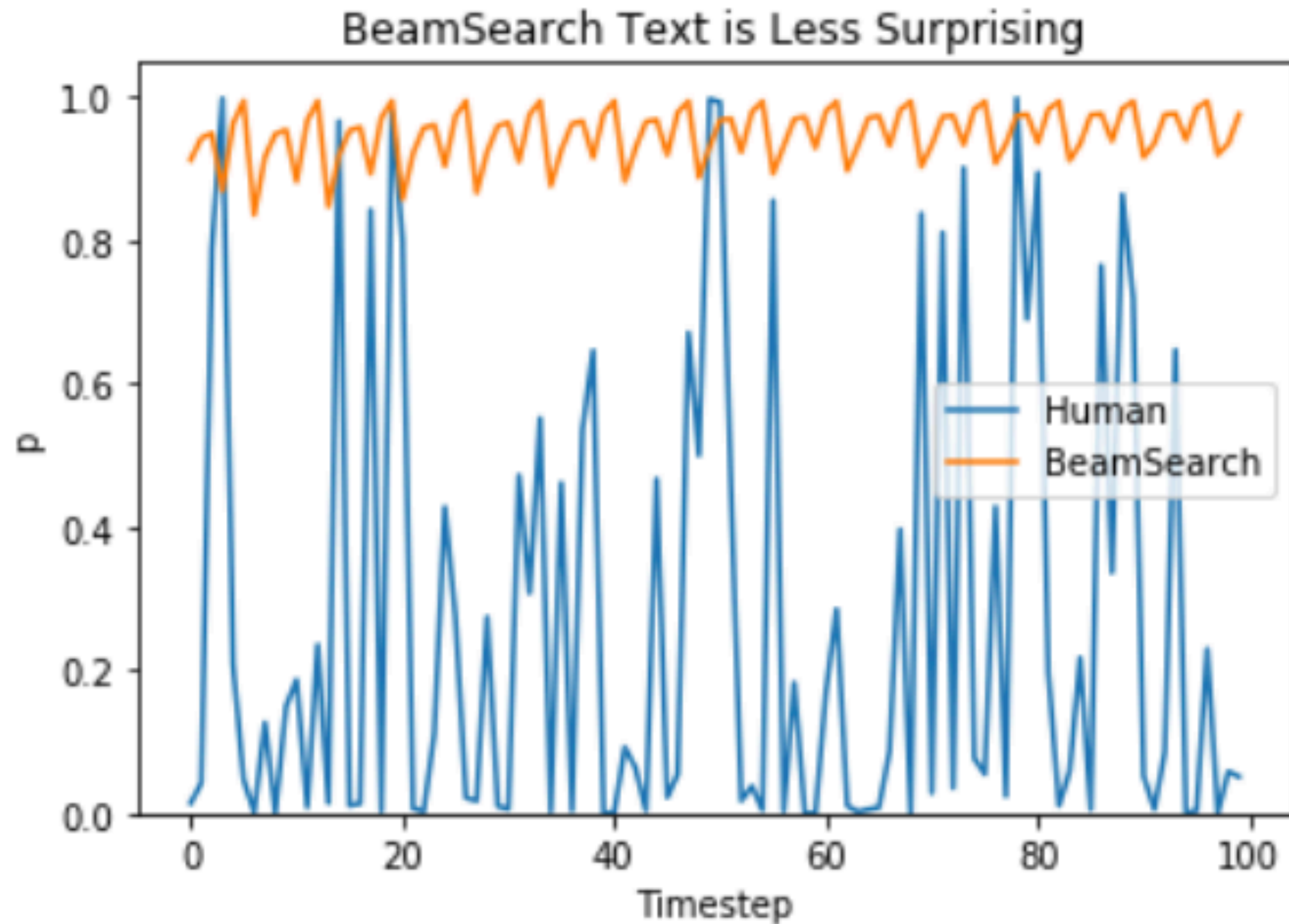
Let us assume a beam size of 2

Keep the 2 best outcomes at each time step

In this example:
("The", "nice") 0.5
("The", "dog") 0.4

Next time step:
("The", "dog", "has") $0.4 * 0.9 = 0.36$
("The", "nice", "woman") $0.5 * 0.4 = 0.2$





Ari Holtzman et al. (2019) plot probability that a model gives versus an estimate of the probability that a human would give. As humans we want generated text to surprise us and not be boring/predictable (depends on the task).

Beam Search Pitfalls

- Beam search can still be very repetitive.
 - Heuristic is to penalize repeated n-grams in the output.
 - Manually set the probability of next words that could create an already seen n-gram to 0
 - n should be greater than 2 or 3
- The choices in beam search may not be very diverse.
 - Similar continuations can happen due to common sub-trees in different branches
- These issues are referred to as **model degeneration**

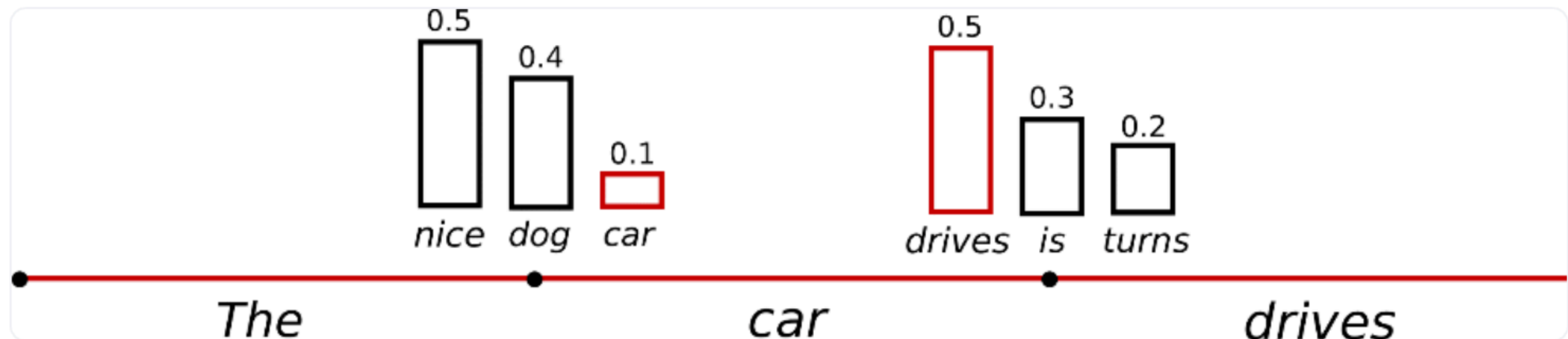
Sampling

- Sampling is represented by the operator $\sim$

- We pick the next word $w_t \sim P(w \mid w_{1:t-1}) = \frac{\exp(\text{logits}(w \mid w_{1:t-1}))}{\sum_{w'} \exp(\text{logits}(w' \mid w_{1:t-1}))}$

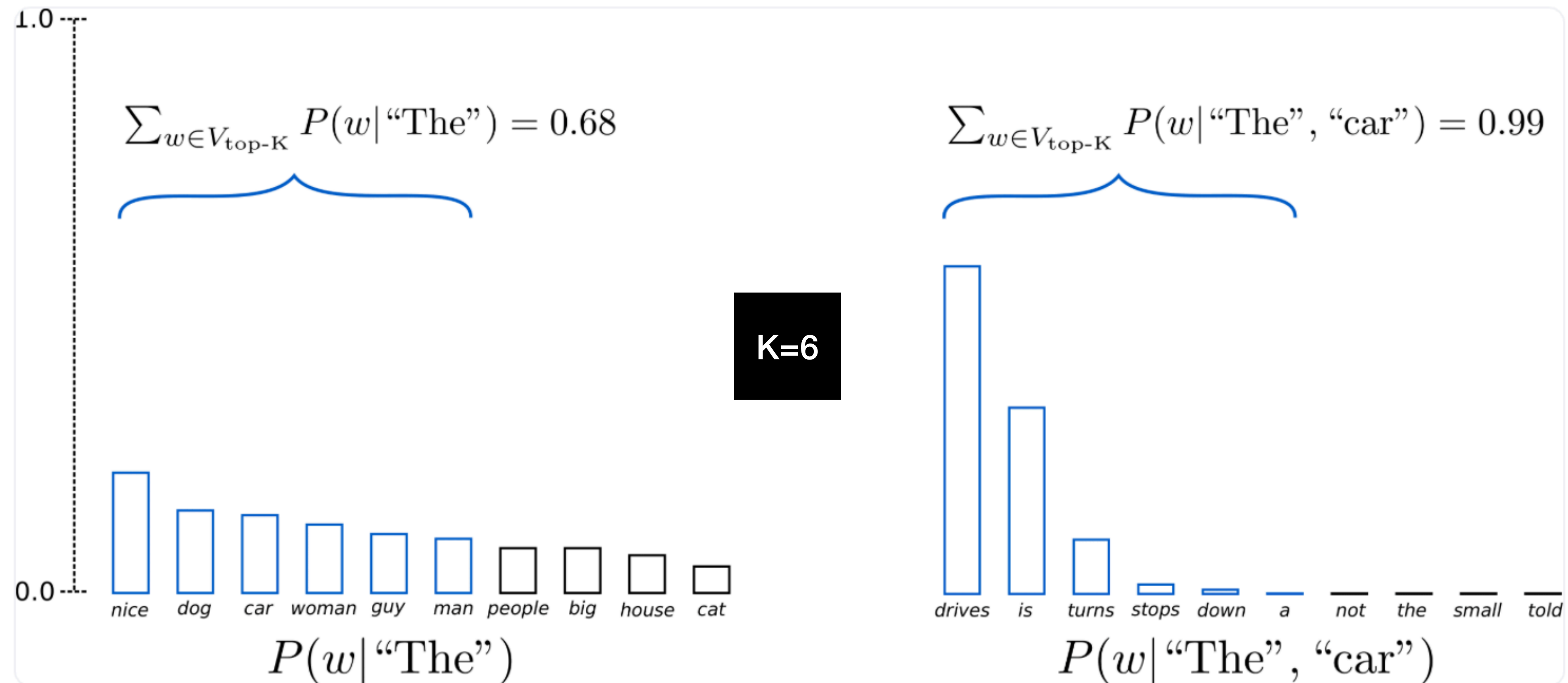
- Generation is no longer *deterministic*.

- Sampling can generate gibberish. Solution: use temperature $\frac{\exp(\text{logits}(w \mid w_{1:t-1})/T)}{\sum_{w'} \exp(\text{logits}(w' \mid w_{1:t-1})/T)}$



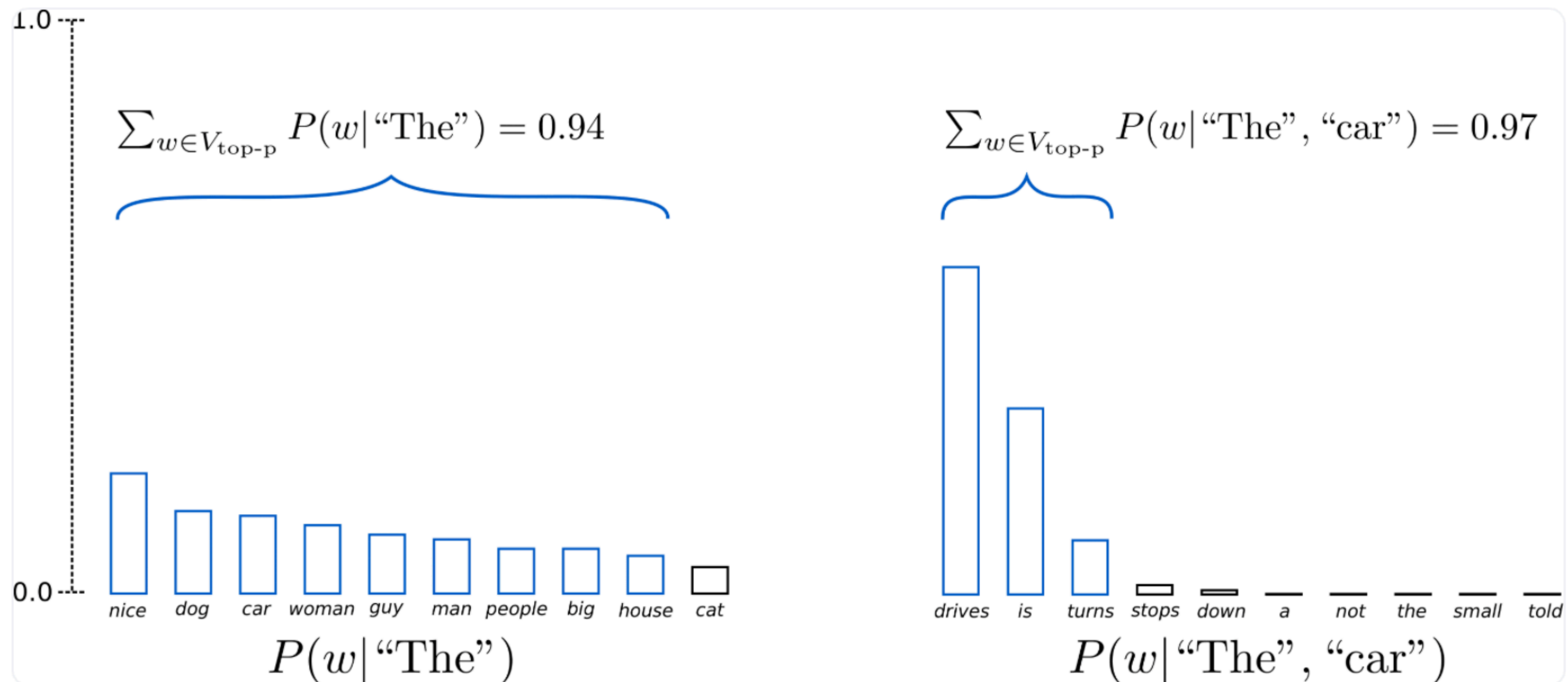
Top-k Sampling

- K most likely next words are filtered and we re-normalize over the K words
- GPT2 showed that this worked better than beam search



Top-p Nucleus Sampling

- Choose the smallest set of words whose cumulative probability exceeds a threshold probability p . The probability mass is redistributed among this set of words.
- The size of the set being sampled from grows and shrinks depending on the probability distribution.

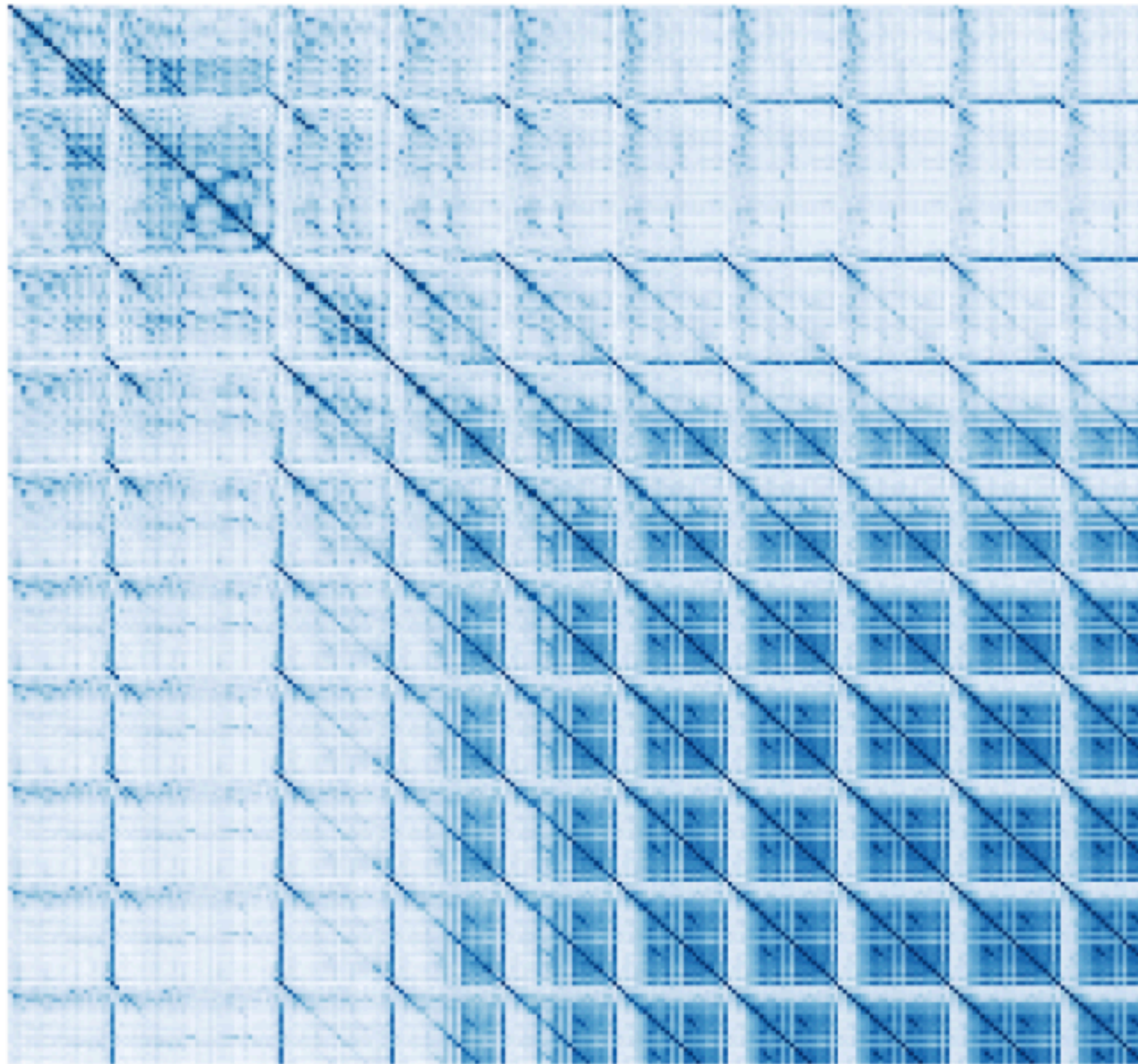


Contrastive Search

- Given a prefix text $\mathbf{x}_{<t}$ select the output next token x_t
- $V^{(k)}$ is the set of top-k predictions from the LM's probability distribution $p_{\theta}(v \mid \mathbf{x}_{<t})$ called the **model confidence**
- $s(\cdot, \cdot)$ is the cosine similarity between two token representations is used to compute the **degeneration penalty**
- The more similar v is to the context the more we see **model degeneration**.
- Combine the two terms using a linear mixture.

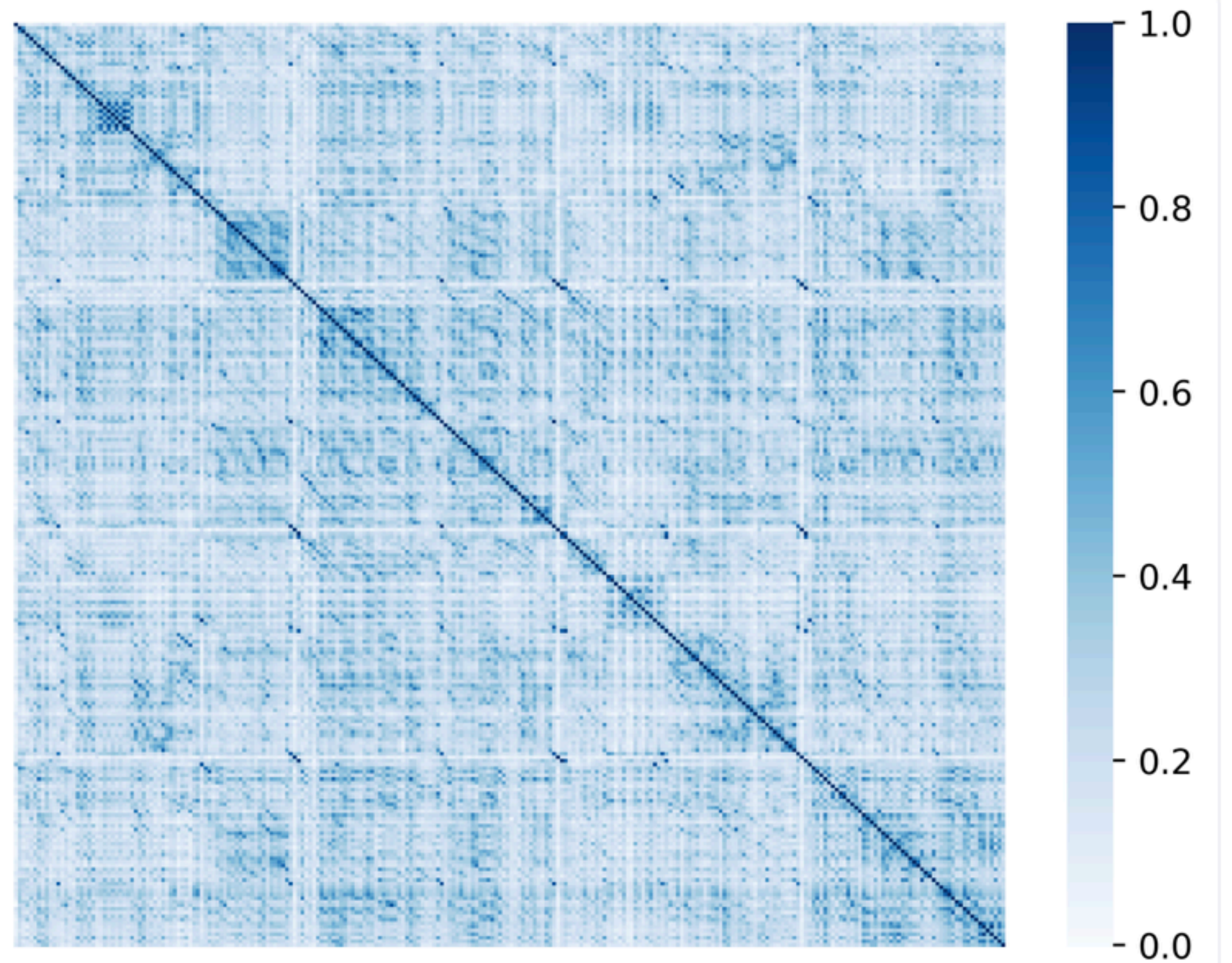
$$x_t = \arg \max_{v \in V^{(k)}} \left\{ (1 - \alpha) \times \underbrace{p_{\theta}(v | \mathbf{x}_{<t})}_{\text{model confidence}} - \alpha \times \underbrace{(\max\{s(h_v, h_{x_j}) : 1 \leq j \leq t - 1\})}_{\text{degeneration penalty}} \right\},$$

Contrastive Search



Greedy Search

Comparison of Similarity Scores



Contrastive Search

Other problems

- **Unreachable subword problem:** there are some subwords for which under no circumstances is it possible to produce a subword (given any context).
- **Mode collapse:** tuning the LM might cause the model parameters to reach a state where Greedy and Sampling based generation produce the same output.
- **Softmax over very large vocabulary sizes:** Vocabulary sizes have reduced since subword segmentation has become the standard way to set up the vocabulary for LMs; However for very large vocabulary sizes, the compute efficiency for softmax might need careful consideration, e.g. use hierarchical softmax.

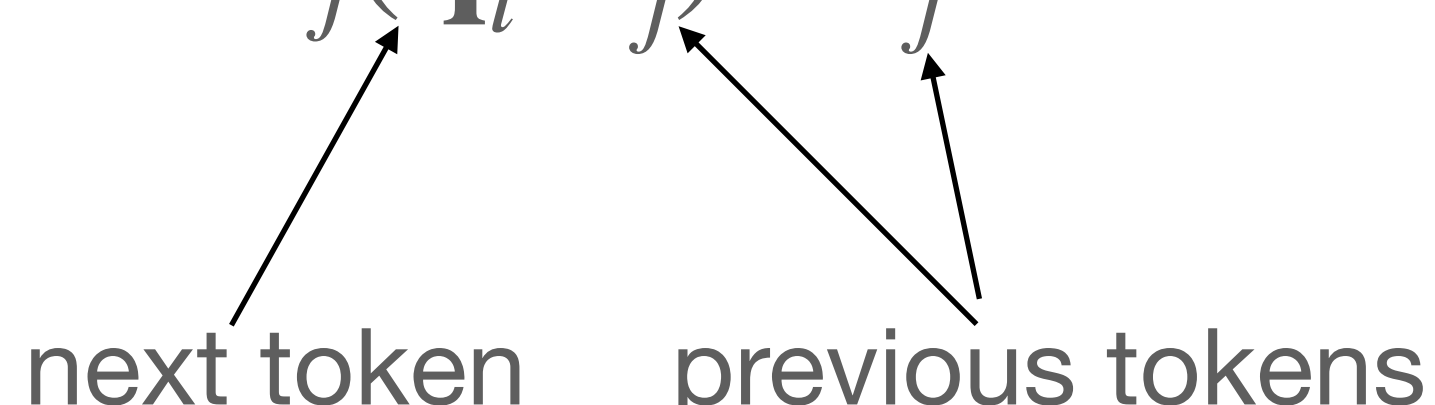
Caching

Self-attention in auto-regressive LMs

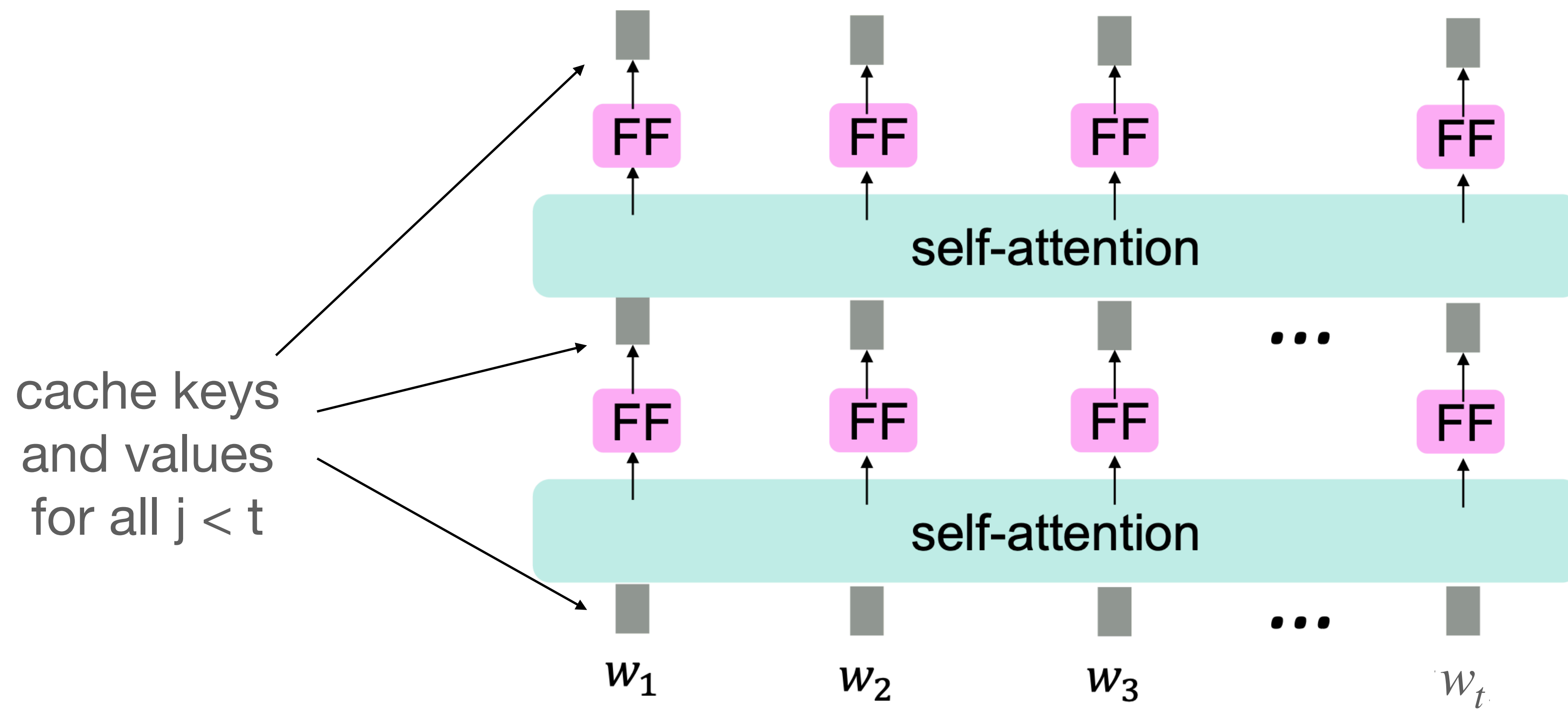
keys, queries and values computed from the history

- Let $\mathbf{w} = (w_1, \dots, w_{t-1})$ be a sequence of tokens, like "Havana is the capital of"
- For each w_i let $\mathbf{x}_i = E\mathbf{w}_i$ where $E \in \mathbb{R}^{d \times |V|}$ is an embedding matrix. V is the vocabulary.
- Let Q, K, V be matrices in $\mathbb{R}^{d \times d}$
 - $\mathbf{q}_i = Q\mathbf{x}_i$
 - $\mathbf{k}_i = K\mathbf{x}_i$
 - $\mathbf{v}_i = V\mathbf{x}_i$

Attention layer output for next token is a weighted sum of values:

$$\mathbf{o}_t = \sum_{j < t} \text{softmax}_j(\mathbf{q}_t^T \mathbf{k}_j) \cdot \mathbf{v}_j$$


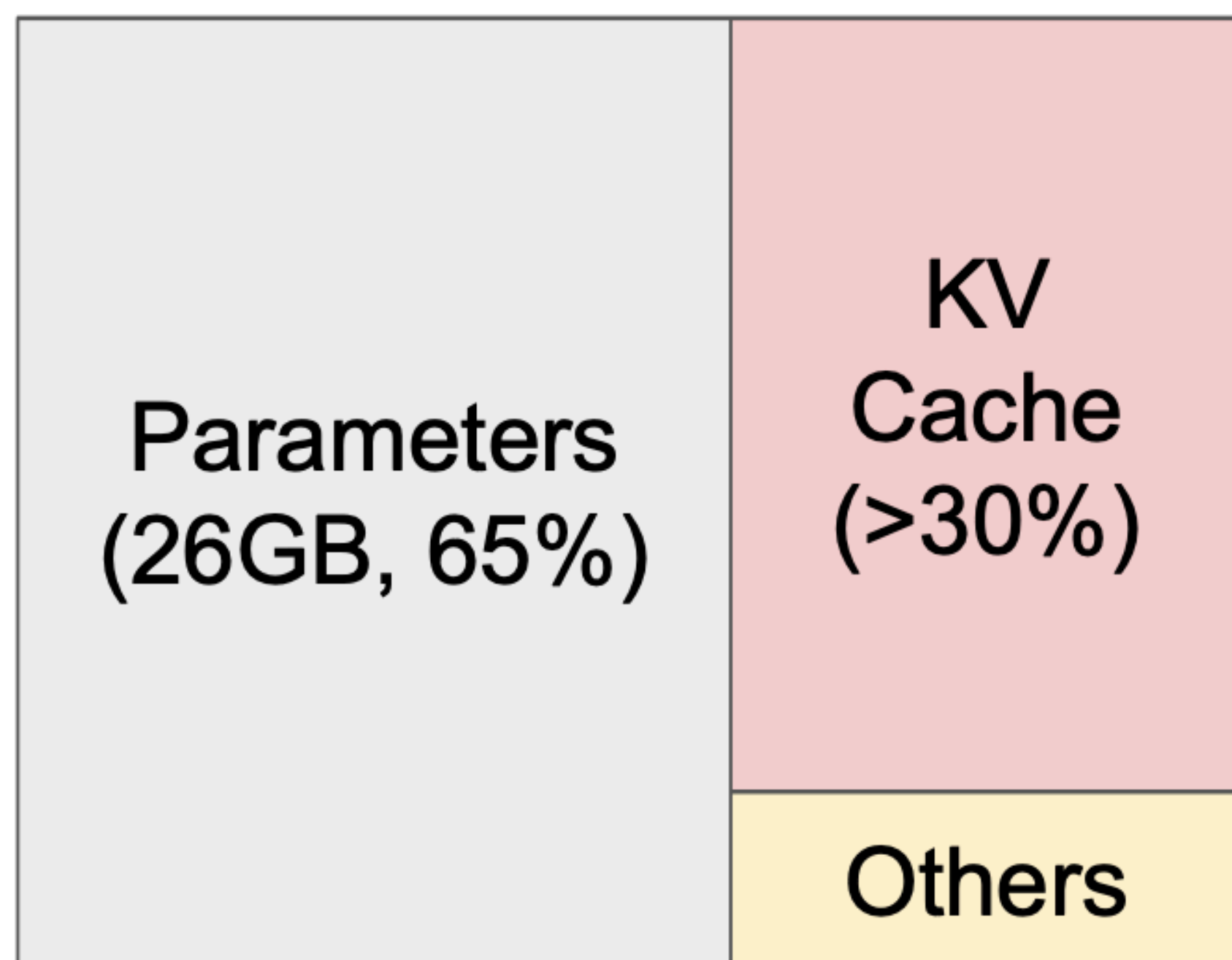
The diagram shows two arrows originating from the text 'next token' at the bottom left. One arrow points to the $\mathbf{q}_t$ term in the equation, and the other points to the $\mathbf{k}_j$ term. A second arrow originates from the text 'previous tokens' at the bottom right and points to the $\mathbf{v}_j$ term.



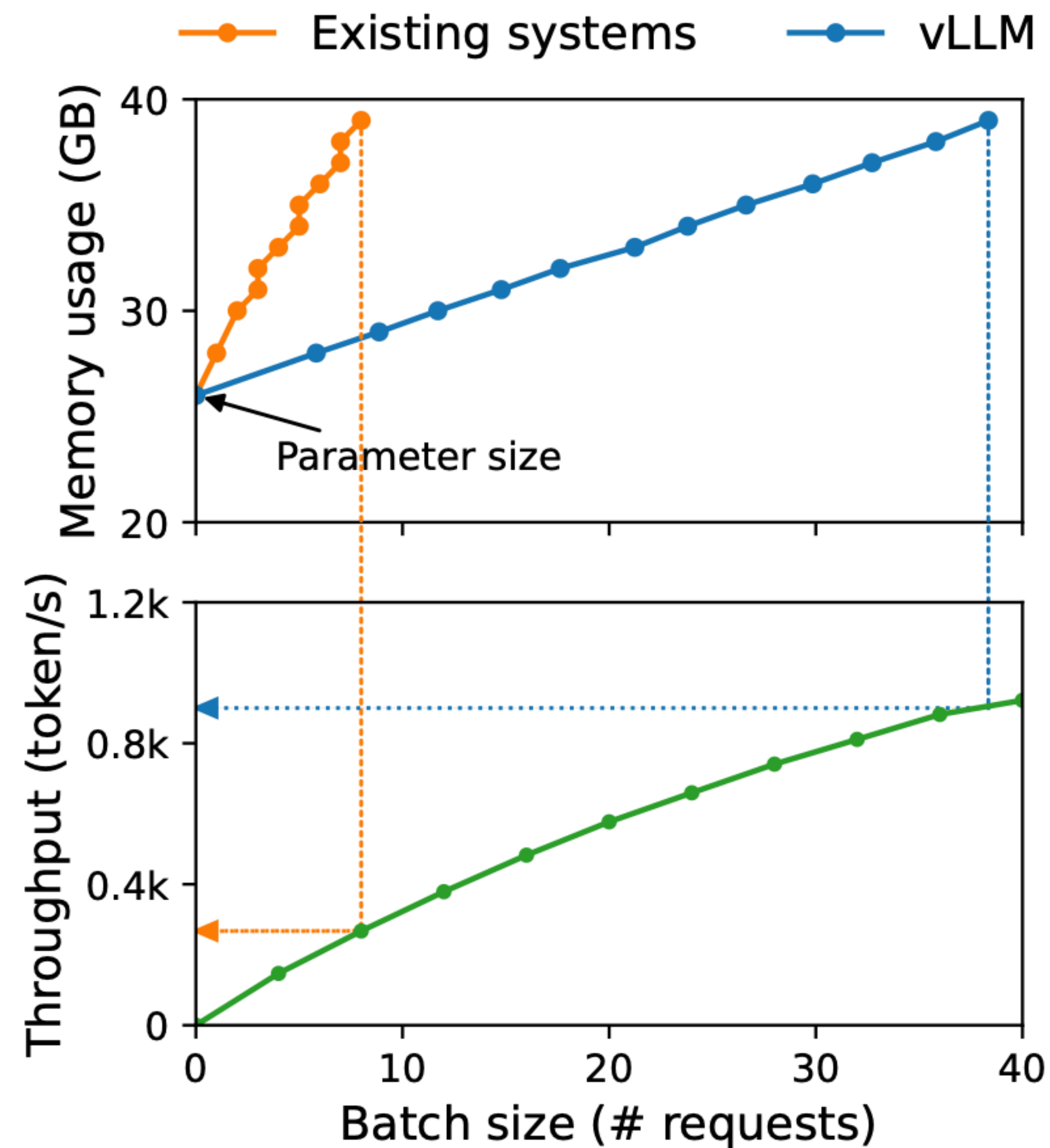
$$\mathbf{o}_t = \sum_{j < t} \text{softmax}_j(\mathbf{q}_t^T \mathbf{k}_j) \cdot \mathbf{v}_j$$

Phases of generation

- Prompt phase
 - all tokens are known
 - generate and store all the key and value tensors
- Autoregressive phase
 - compute the probability of the next token using cached tensors
 - add the key and value tensor to the cache



NVIDIA A100 40GB



<https://arxiv.org/pdf/2309.06180>

Paged Attention for efficient memory usage

implemented in the vllm inference engine

